Task 02: Packet Sniffing, Firewall Configuration, and Vulnerability Scanning

1. Introduction:

This document presents the implementation of three important cybersecurity tasks: **packet sniffing, firewall configuration, and vulnerability scanning**. These tasks help security professionals monitor network traffic, control access to systems, and identify potential security weaknesses.
The objective of this exercise is to gain practical experience using **Scapy for packet analysis, iptables for firewall configuration, and OpenVAS for vulnerability scanning**.
These tools are widely used by **Security Operations Center (SOC) analysts and penetration testers** to monitor networks and improve system security

2. Objective:

The main objectives of this task are:

- To capture and analyze network packets using **Scapy**
- To configure a basic **Linux firewall using iptables**
- To perform **vulnerability scanning using OpenVAS**
- To analyze security weaknesses in a network environment

3. Tools Used

The following tools were used during this task:

Tool	Purpose
Kali Linux	Security testing environment
Python	Script development
Scapy	Packet sniffing and analysis
Matplotlib	Protocol distribution chart

Tool	Purpose
iptables	Firewall configuration
OpenVAS (Greenbone)	Vulnerability scanning

4. Project Structure

The following directory structure was created to organize the project files.

```
cyart-task02/
│
├── packet_sniffer.py
├── firewall_rules.txt
├── protocol_chart.png
├── report_screenshots/
```

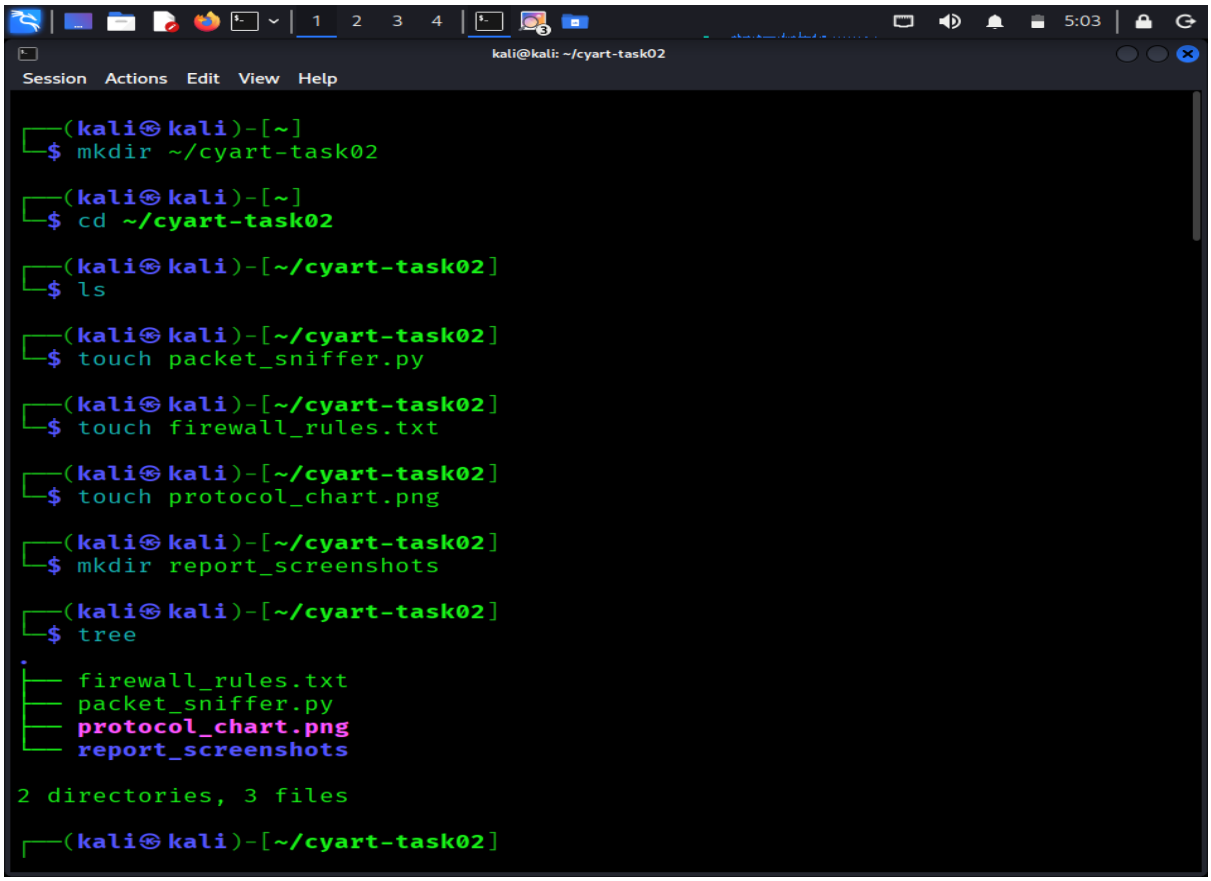


Figure 1: Project Folder Structure

5.Packet Sniffing Using Scapy

Packet sniffing is the process of **capturing network packets for analysis**. This helps security analysts understand network communication and detect suspicious traffic.
In this task, **Scapy**, a powerful Python-based packet manipulation tool, was used to capture packets from the local machine

5.1 Installing Scapy

Scapy was installed using the following command:
sudo apt install python3-scapy

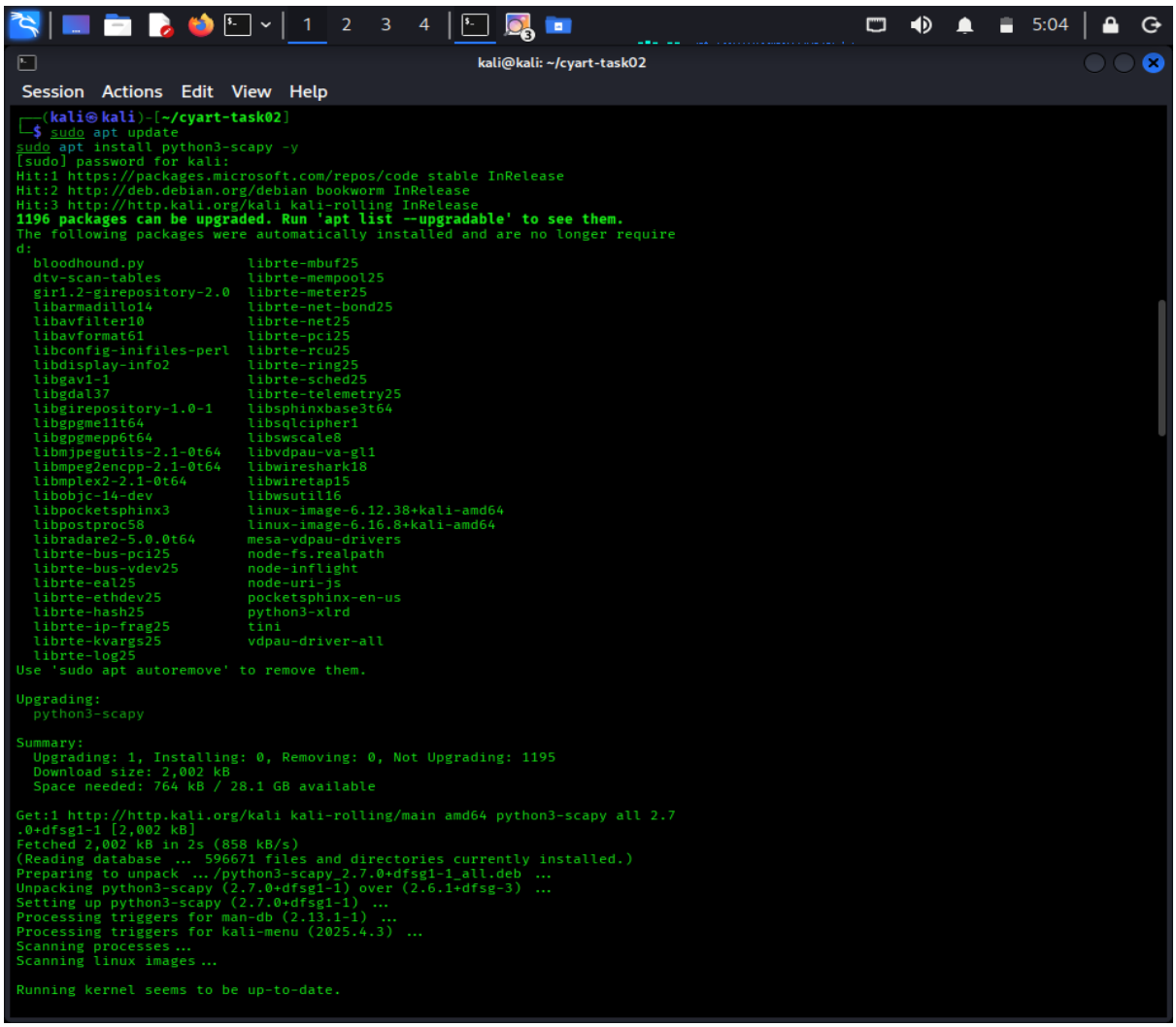


Figure 2: Scapy Installation

5.2 Packet Sniffer Python Script

A Python script named **packet_sniffer.py** was created to capture **100 packets** from the network interface.

The script identifies different protocols such as:

- TCP
- UDP
- ICMP

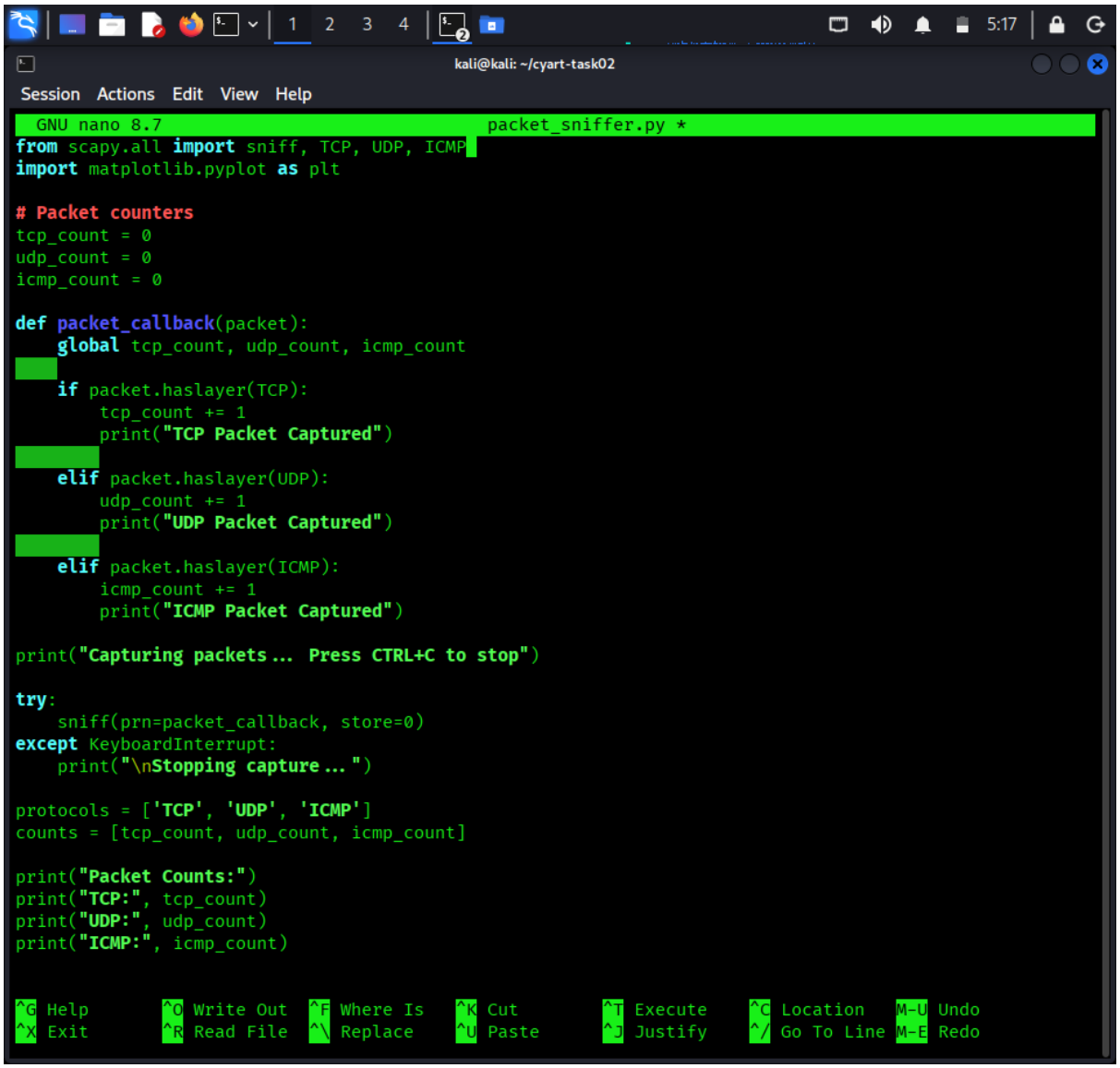
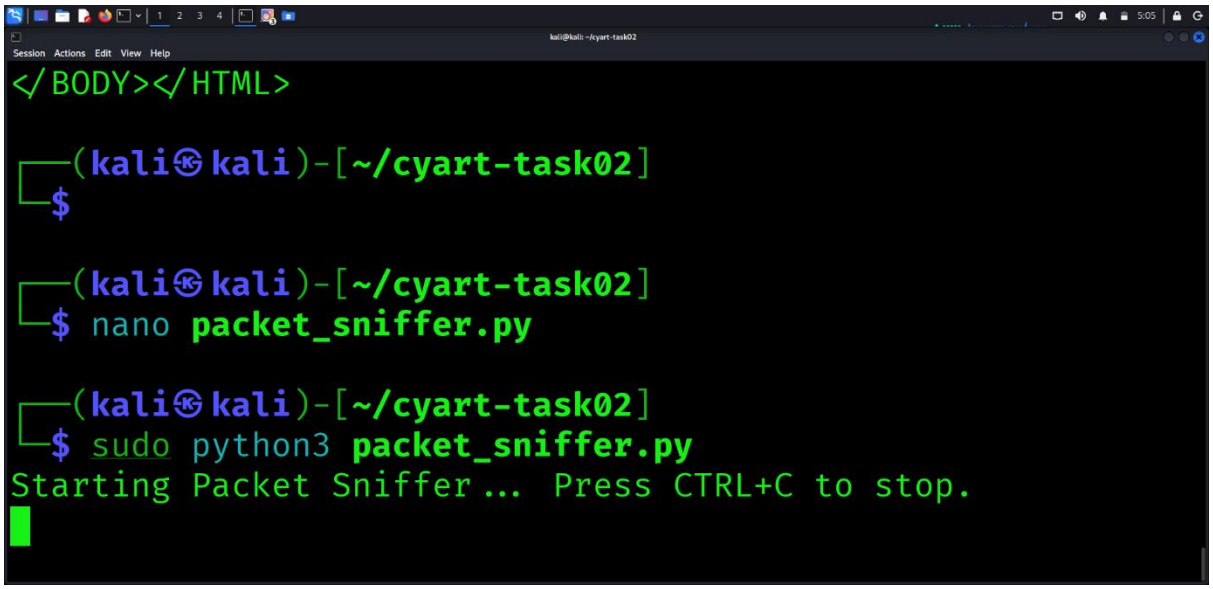


Figure 3: Packet Sniffer Python Code

5.3 Capturing Network Traffic

The packet sniffer was executed using the following command:

sudo python3 packet_sniffer.py
The script captured packets and printed protocol information to the terminal.



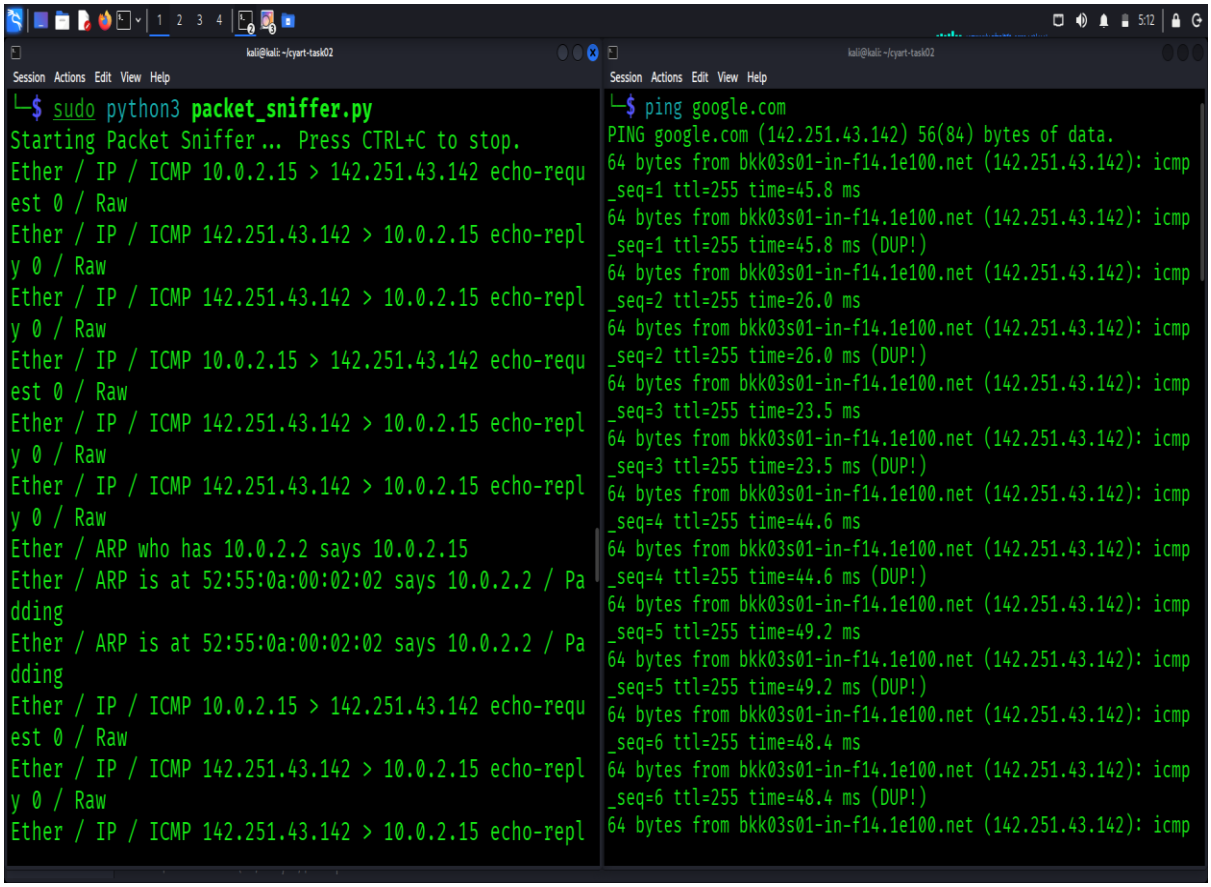


Figure 4: Packet Sniffer Started , Figure 5: Packets Captured

6. Protocol Distribution Analysis

After capturing packets, the protocols were counted and visualized using **Matplotlib**.
The distribution of captured protocols included:

- TCP packets
- UDP packets
- ICMP packets

A **bar chart** was generated to visualize protocol distribution.

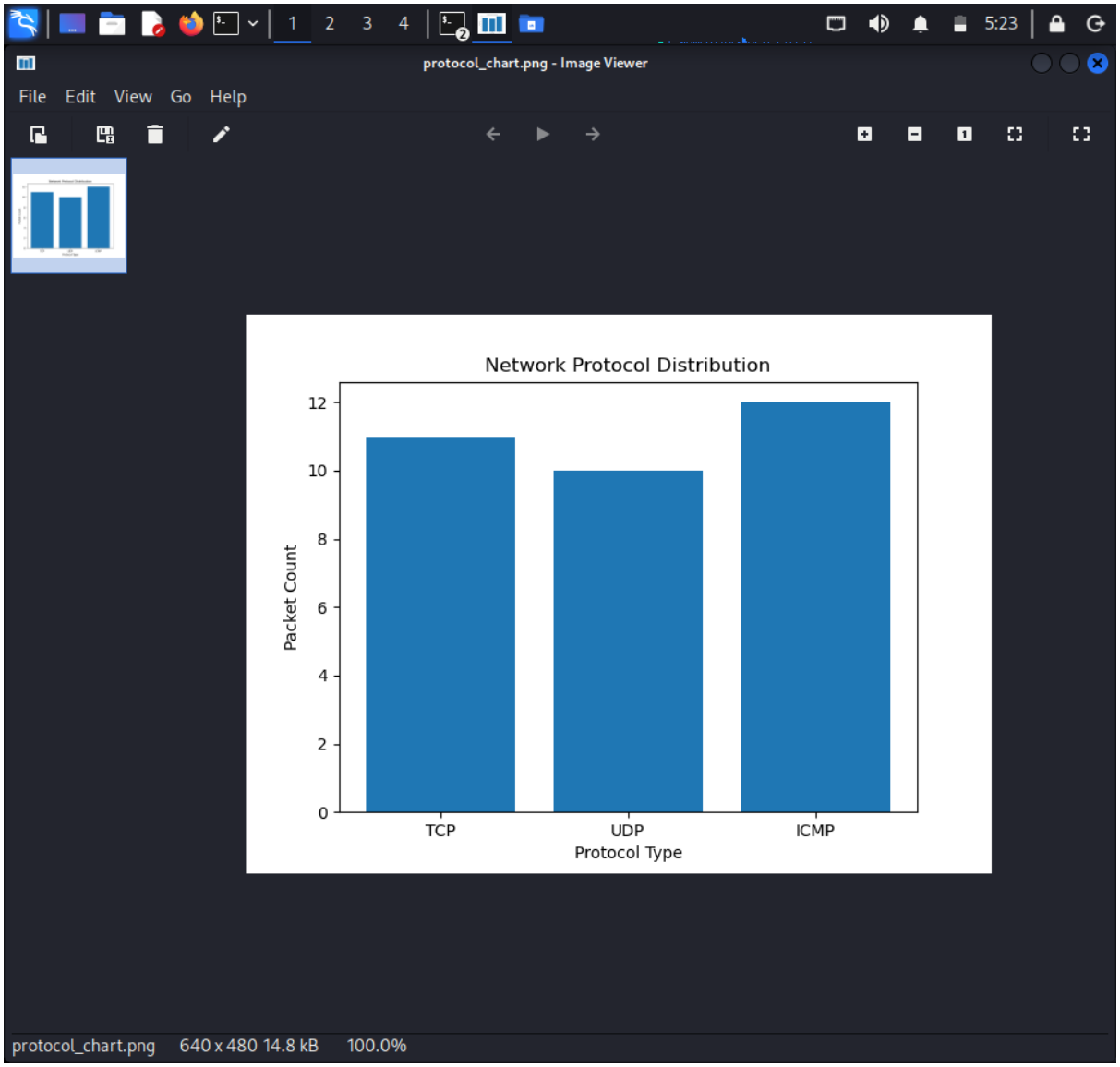


Figure 6: Protocol Distribution Chart

7. Firewall Configuration Using iptables

A basic firewall was configured using **iptables** to control incoming network traffic.
The firewall rules were designed to:

- Allow **SSH traffic (port 22)**
- Allow **HTTP traffic (port 80)**
- Block all other incoming traffic

7.1 Initial Firewall Rules

Before configuration, existing firewall rules were checked using:
sudo iptables -L

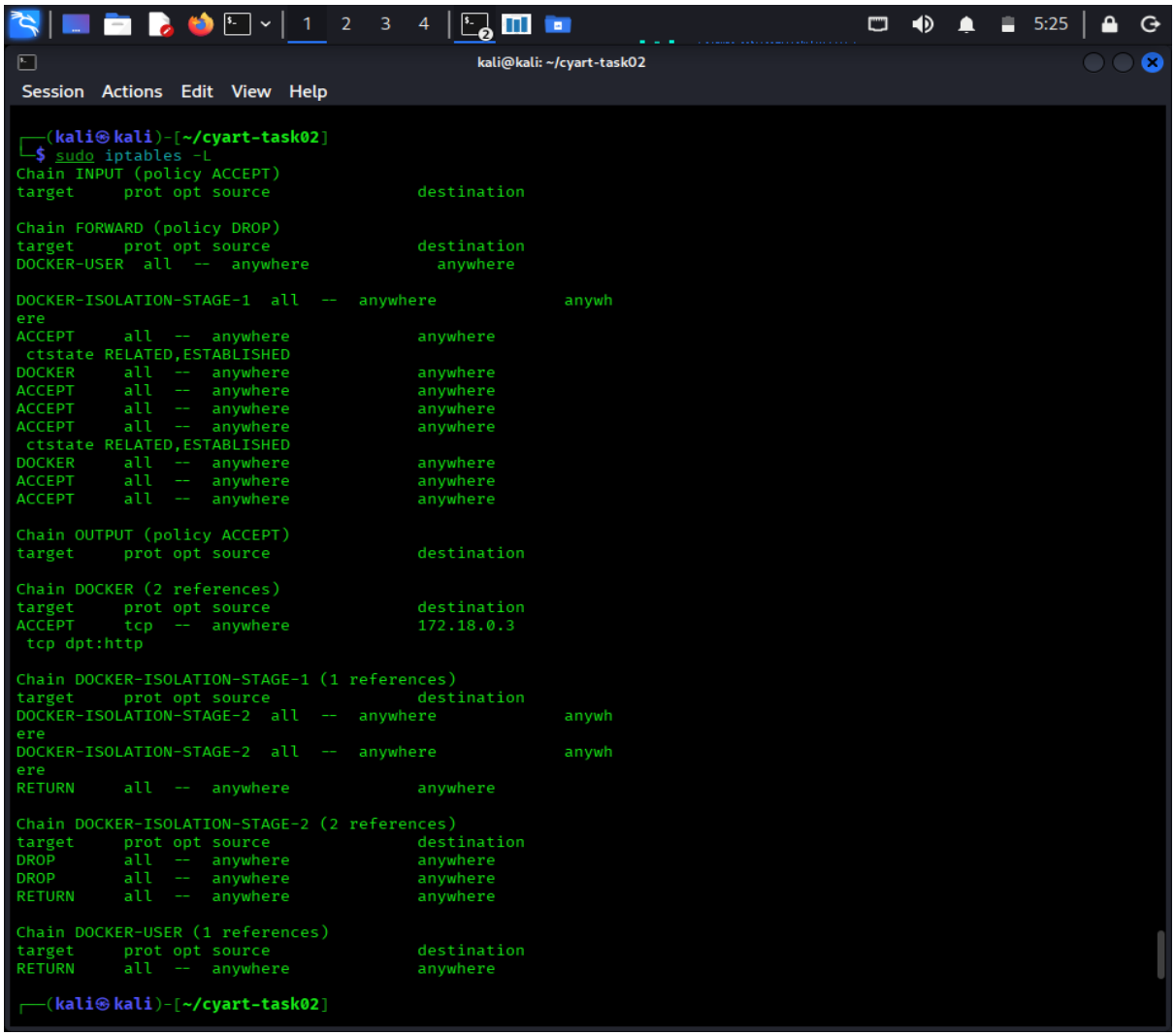


Figure 7: Initial Firewall Rules

7.2 Allowing SSH Traffic

The following rule allows SSH connections:
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

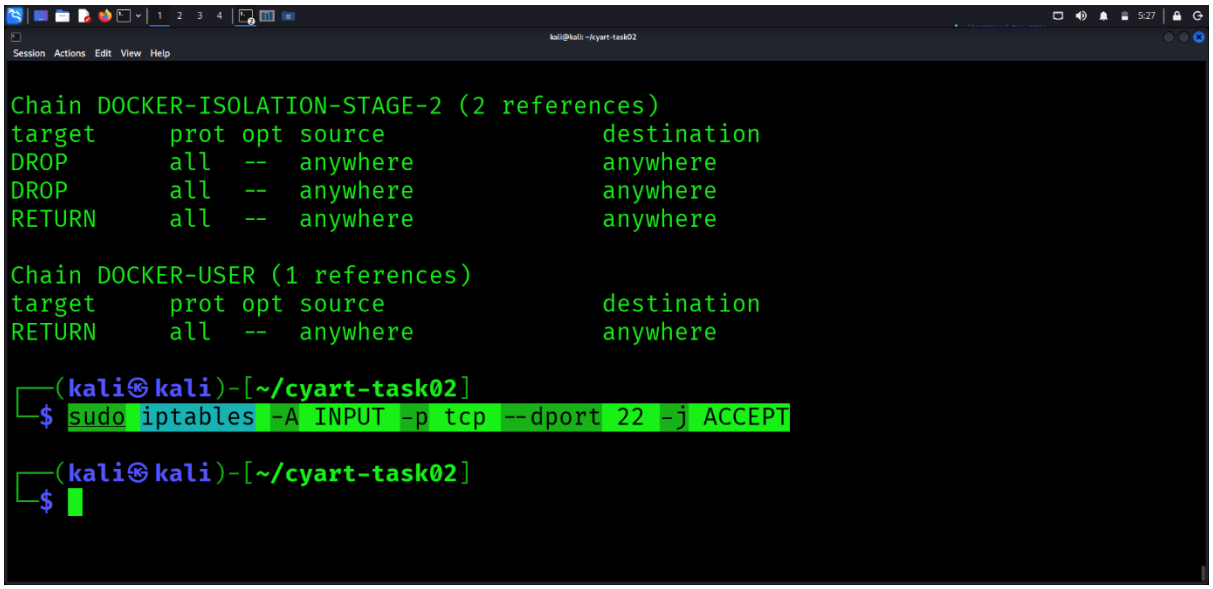


Figure 8: Allow SSH Rule

7.3 Allowing HTTP Traffic

The following rule allows HTTP traffic:
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

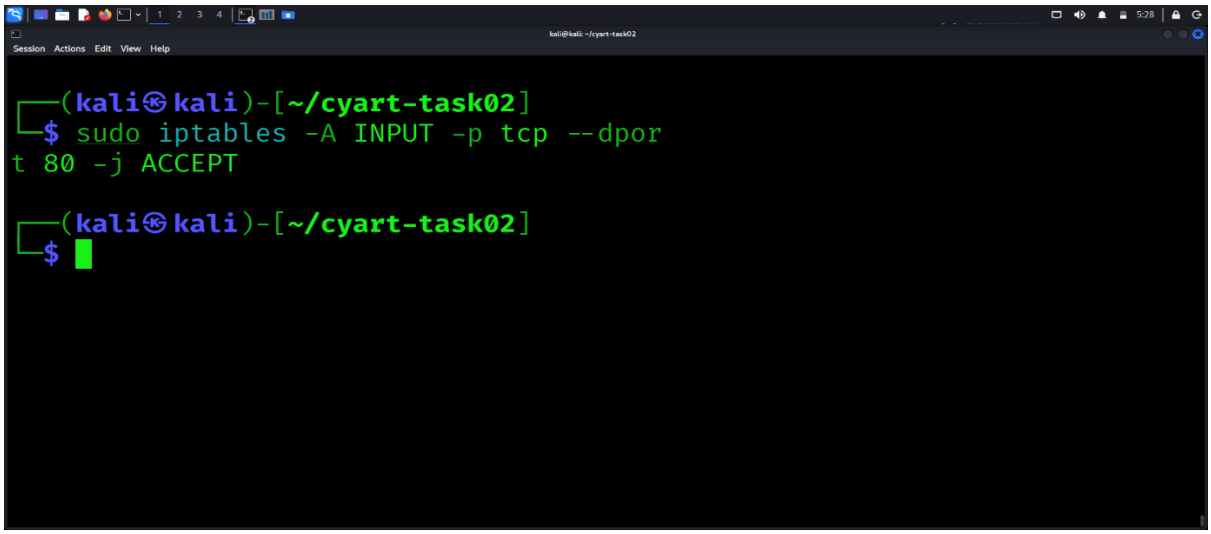


Figure 9: Allow HTTP Rule

7.4 Blocking Other Traffic

All other incoming traffic was blocked using:
sudo iptables -A INPUT -j DROP

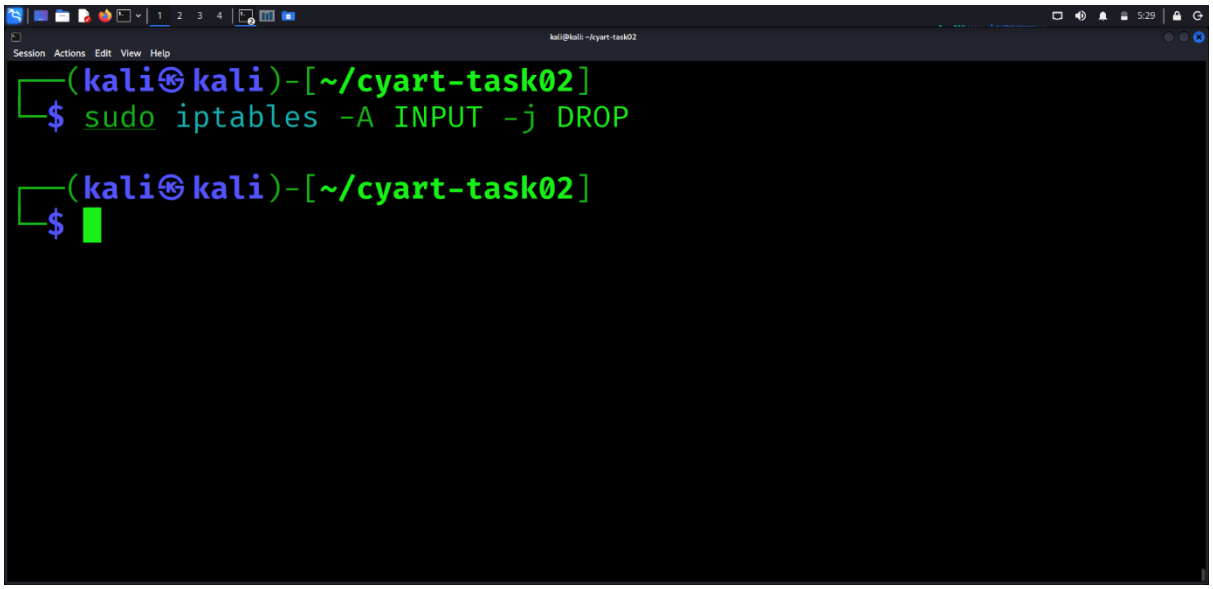


Figure 10: Block Other Traffic

8. Vulnerability Scanning Using OpenVAS

OpenVAS (Open Vulnerability Assessment System) is a powerful vulnerability scanner used to detect security weaknesses in systems.

8.1 Installing OpenVAS

OpenVAS was installed and configured using Kali Linux tools.

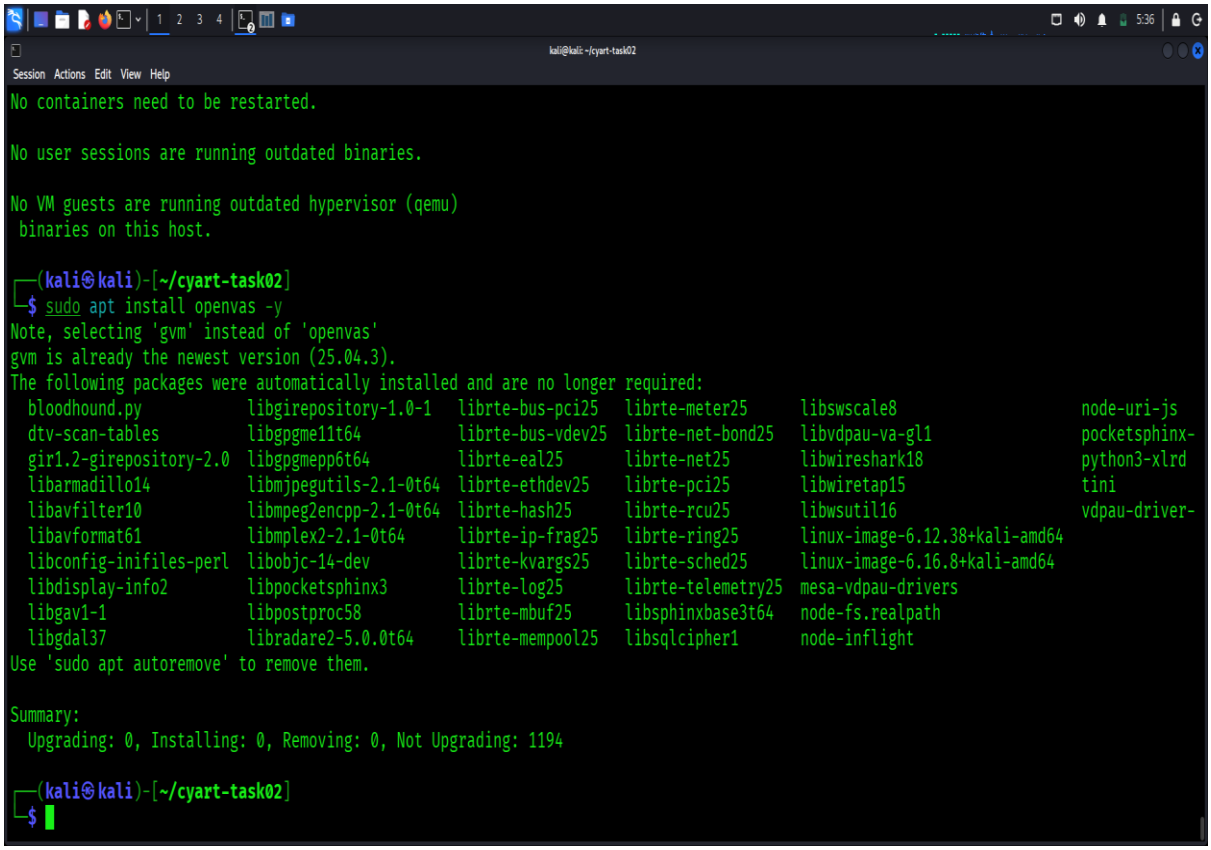


Figure 11: OpenVAS Installation

8.2 OpenVAS Setup

After installation, the OpenVAS services were started and the web interface was accessed through a browser.

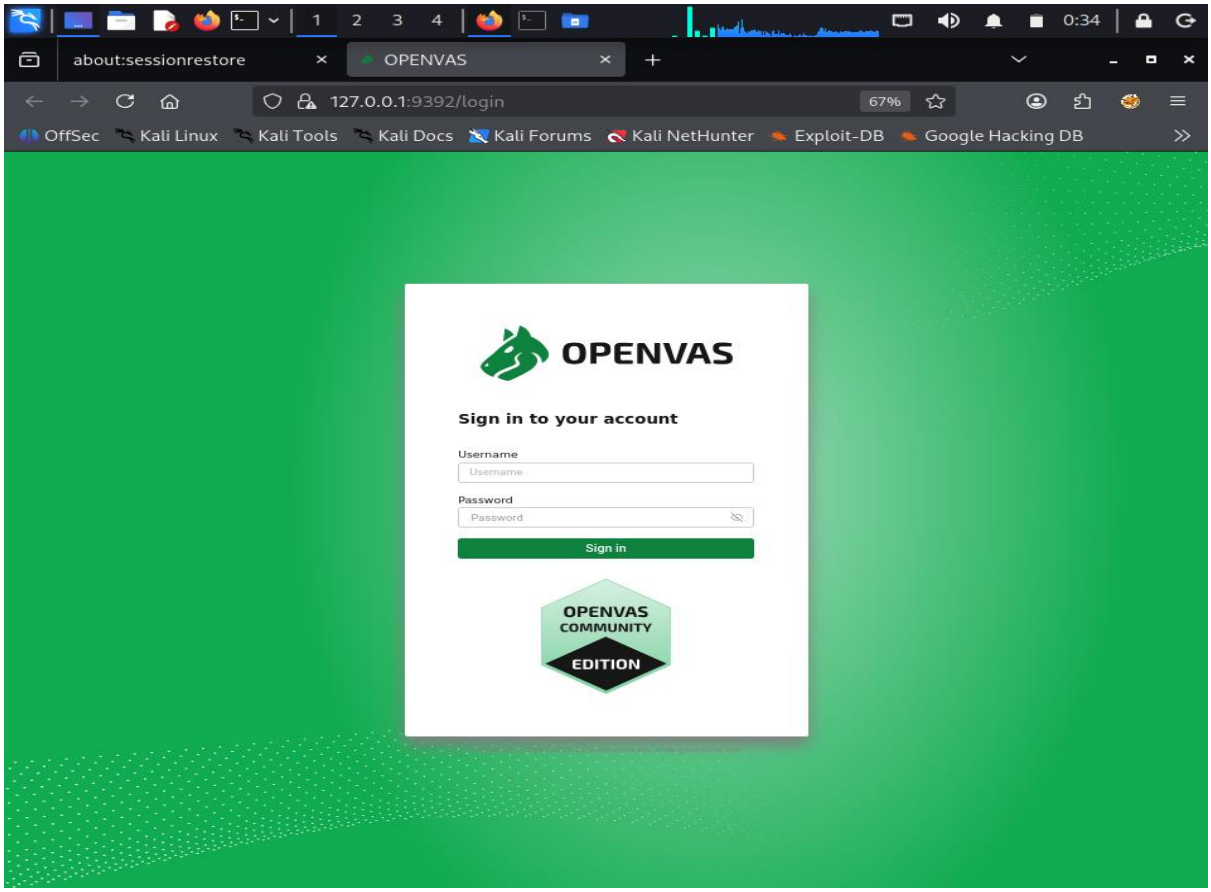


Figure 12: OpenVAS Login Page

8.3 Running a Vulnerability Scan

A target system was created and scanned to identify potential vulnerabilities.

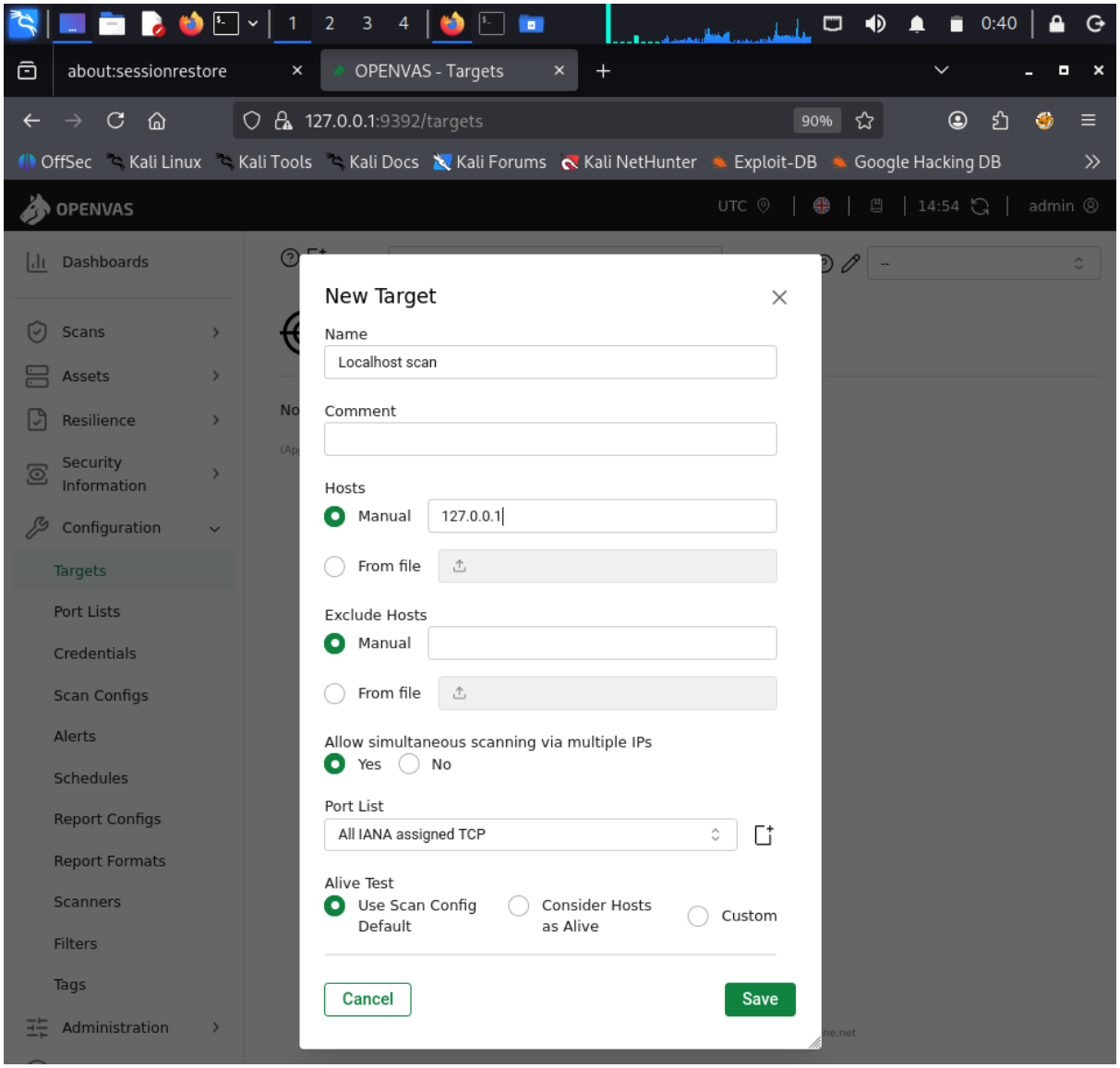


Figure 14: Target Configuration

The scan results help security teams identify weaknesses such as:

- outdated software
- insecure configurations
- known vulnerabilities

9. Key Learnings

During this task, several important cybersecurity concepts were learned:

- How to capture and analyze network packets using **Scapy**
- How firewall rules control network traffic
- How to configure a basic firewall using **iptables**

- How vulnerability scanners identify security weaknesses
- How visualizations such as charts help analyze network data

10. Challenges Faced

Some challenges encountered during this task included:

- Configuring firewall rules correctly
- Ensuring proper packet capture permissions
- Installing and configuring OpenVAS services
- Interpreting vulnerability scan results

These challenges were resolved by referring to official documentation and troubleshooting guides.

11. Conclusion

This task provided hands-on experience in **network monitoring, firewall configuration, and vulnerability assessment**.
By using tools such as **Scapy, iptables, and OpenVAS**, it was possible to analyze network traffic, enforce access control policies, and identify security vulnerabilities.
These skills are essential for cybersecurity professionals working in **SOC environments and penetration testing roles**.